

DIGITAL ELECTRONICS & LOGIC DESIGN

Module 1 Reference Handbook: Fundamentals, ICs & Binary Codes

Syllabus Weight: 5
Hours

Target: Engineering Lecture Notes & Exam
Study Guide

Core Focus: Signals, Logic Families, Triggering &
Number Codes

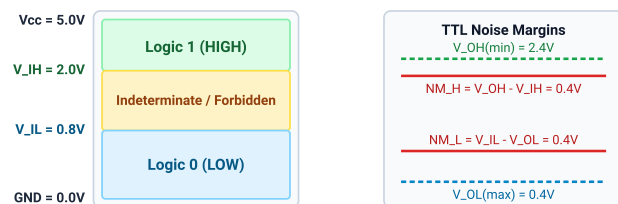
1.1 DIGITAL VERSUS ANALOG SIGNALS

Signals carry information by representing physical variables electrically. In electronics systems, signals fall into two main categories: continuous-time analog signals and discrete-time digital signals.

Parameter	Analog Signals	Digital Signals
Representation	Continuous range of infinite voltage/current values over time.	Discrete logic states (typically binary '0' and '1').
Noise Immunity	Low; susceptible to noise distortion, signal attenuation, and drift.	High; broad voltage tolerance thresholds reject environmental noise.
Storage & Processing	Complex; difficult to store without analog signal degradation.	Efficient; easily stored in modern high-density semiconductor memory.
Transmission	Requires high-precision linear amplification and filtering.	Regenerative repeaters restore ideal pulse shapes over long paths.
Typical Examples	Raw acoustic sound waves, thermistor temperature voltages, RF carrier waves.	CPU bus signals, digital PCM audio, USB/PCIe data lines, flash memory.

1.2 LOGIC LEVEL DIAGRAM & VOLTAGE STANDARDS

Digital logic maps continuous electrical voltages into binary logic HIGH (1) and LOW (0). To account for signal noise, component tolerances, and transmission loss, IC standards define specific voltage threshold regions.



- **V_{OH} (min):** Minimum output voltage guaranteed by a logic gate for Logic 1 (2.4V in TTL).
- **V_{IH} (min):** Minimum input voltage reliably recognized as Logic 1 (2.0V in TTL).
- **V_{IL} (max):** Maximum input voltage reliably recognized as Logic 0 (0.8V in TTL).
- **V_{OL} (max):** Maximum output voltage produced for Logic 0 (0.4V in TTL).
- **High-State Noise Margin:** $NM_H = V_{OH(min)} - V_{IH(min)}$
- **Low-State Noise Margin:** $NM_L = V_{IL(max)} - V_{OL(max)}$

1.3 DIGITAL INTEGRATED CIRCUITS (ICS) & LOGIC FAMILIES

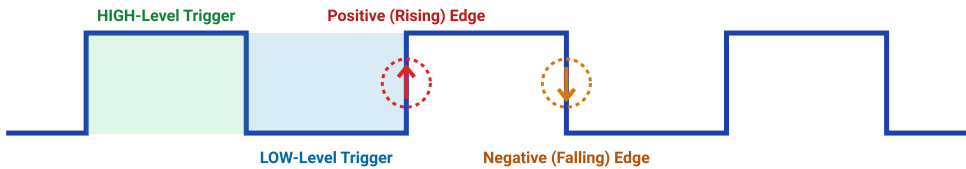
Digital circuits are fabricated on silicon monolithic chips categorized by integration density and physical component technology.

Integration Scale Classification			Dominant Logic IC Families	
Category	Gate Density	Typical Applications		
SSI	< 10 gates	Basic logic gates (7400 NAND, 7402 NOR)		
MSI	10 – 100 gates	Decoders, Adders, Multiplexers, Counters		
LSI	100 – 10,000 gates	8-bit CPUs, RAM, ROM memories		
VLSI	10k – 100k gates	16/32-bit Microprocessors, DSPs		

		• TTL (Transistor-Transistor Logic): Utilizes Bipolar Junction Transistors (BJTs). Standard 5V power supply. Fast switching speed, high drive capacity, but draws higher static power. (e.g., 74HC/74LS series).
		• CMOS (Complementary Metal-Oxide Semiconductor): Employs complementary PMOS and NMOS FET pairs. Wide supply range (3V–15V). Extremely low static power consumption and high noise immunity; dominant in modern IC design.
		• ECL (Emitter-Coupled Logic): Non-saturating BJT logic designed for ultra-high-speed operations; high power

1.4 CLOCK TRIGGERING SYSTEMS

Synchronous sequential circuits rely on a continuous clock pulse signal to coordinate internal state changes. Triggering mechanisms determine exactly when a flip-flop or register responds to its data inputs.



Level-Sensitive Triggering

The circuit output continuously updates or responds as long as the clock signal remains at a specific logic voltage level.

- **High-Level Trigger:** Circuit active when **CLK = 1**.
- **Low-Level Trigger:** Circuit active when **CLK = 0**.
- **Drawback:** Susceptible to race conditions during long clock high periods (transparent latch behavior).

Edge-Sensitive Triggering

The circuit state transitions occur *only* during the rapid voltage transition threshold between binary logic states.

- **Positive-Edge (Rising):** Triggered on **0 → 1** transition.
- **Negative-Edge (Falling):** Triggered on **1 → 0** transition.
- **Advantage:** Eliminates clock race conditions; standard timing mechanism for flip-flops and registers.

1.5 DIGITAL SYSTEM APPLICATIONS

Computing & Memory

Microprocessors, microcontrollers, graphics processing units (GPUs), and volatile/non-volatile memory chips (DRAM, NVMe SSDs) execute complex logical operations and store binary data.

Telecommunications

Digital communication networks utilize Pulse Code Modulation (PCM), packet switching, 5G baseband processing, and fiber-optic encoders to transmit error-free data over vast distances.

Embedded Control

Automotive ECUs, industrial Robotics, PLCs, and medical instrumentation use digital sensors, DSPs, and PWM motor drivers to control physical processes with high accuracy.

1.6 DIGITAL CODES AND NUMBER SYSTEM CONVERSIONS

1.6.1 Decimal, Binary, Octal, and Hexadecimal Codes

A positional number system represents numbers using a base (radix) r . Any real number can be evaluated using a summation of positional digit weights:

General Positional Formula:
$$N = d_{n-1} r^{n-1} + \dots + d_1 r^1 + d_0 r^0 + d_{-1} r^{-1} + d_{-2} r^{-2} + \dots$$

System	Radix (r)	Valid Symbols / Digits	Positional Weight Sequence	Example Representation
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	..., 10^2 (100), 10^1 (10), 10^0 (1), . 10^{-1} (0.1)	(245.5)₁₀
Binary	2	0, 1	..., 2^3 (8), 2^2 (4), 2^1 (2), 2^0 (1), . 2^{-1} (0.5)	(11110101.1)₂
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	..., 8^2 (64), 8^1 (8), 8^0 (1), . 8^{-1} (0.125)	(365.4)₈
Hexadecimal	16	0–9, A(10), B(11), C(12), D(13), E(14), F(15)	..., 16^2 (256), 16^1 (16), 16^0 (1), . 16^{-1} (0.0625)	(F5.8)₁₆

Universal Conversion Methodology

Base- r to Decimal Conversion:

Multiply each digit by its positional weight r^i and sum all terms together.

(1011.1)₂ = $1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1}$ = 7.5₁₀

Decimal to Base- r Conversion:

Divide the integer part successively by radix r (collect remainders). Multiply fractional part by r (collect integers).

25₁₀ → /2 = 12 r1, /2 = 6 r0, /2 = 3 r0, /2 = 1 r1

1.6.2 BCD Code (Binary Coded Decimal / 8421 Code)

BCD represents each individual decimal digit (0 through 9) using a distinct 4-bit binary code group. It is widely used in numeric digital readouts (e.g., 7-segment displays, digital multimeters).

Decimal	8421 BCD	Decimal	8421 BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Invalid BCD States & Comparison

- **Forbidden States:** Since 4 bits yield 16 states (2^4), the 6 binary combinations from **1010** (10) to **1111** (15) are **invalid / illegal** in BCD.
- **BCD vs Pure Binary:** BCD is less memory-efficient. For example:
 $(185)_{10} = (0001\ 1000\ 0101)_{BCD}$ [12 bits]
 $(185)_{10} = (10111001)_2$ [8 bits in pure binary]

1.6.3 Excess-3 Code

Excess-3 is an unweighted, non-positional BCD code formed by adding 3 (**0011₂**) to each decimal digit's standard 8421 BCD equivalent.

Self-Complementing Property

Excess-3 is a **self-complementing code**. Inverting all bits (1's complement) of an Excess-3 code word directly produces the Excess-3 code for the 9's complement of the corresponding decimal digit.

Example: Decimal 4 is **0111** in Excess-3. Inverting bits gives **1000**, which represents Decimal 5 ($9 - 4 = 5$). This property greatly simplifies arithmetic subtraction in hardware logic ALUs.

Decimal	8421 BCD	Excess-3 (BCD + 3)
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111

1.6.4 Gray Code (Reflected Binary Code)

Gray code is an unweighted, non-positional code where two successive numerical values differ by **only one bit position** (Unit Distance Code). It prevents spurious transient errors in mechanical rotary encoders and analog-to-digital converters.

Binary → Gray Conversion

1. The Most Significant Bit (MSB) of Gray code equals the MSB of binary: $G_n = B_n$.
2. Exclusively-OR (XOR) adjacent binary bits to obtain successive Gray bits:
 $G_i = B_{i+1} \oplus B_i$

Binary (1101) → Gray: $G_3=1$, $G_2=1\oplus1=0$, $G_1=1\oplus0=1$, $G_0=0\oplus1=1 \Rightarrow (1011)_{Gray}$

Gray → Binary Conversion

1. The MSB of binary equals the MSB of Gray code: $B_n = G_n$.
2. XOR the generated binary bit with the next lower Gray code bit:
 $B_i = B_{i+1} \oplus G_i$

Gray (1011) → Binary: $B_3=1$, $B_2=1\oplus0=1$, $B_1=1\oplus1=0$, $B_0=0\oplus1=1 \Rightarrow (1101)_2$

1.6.5 EXAMPLES OF CODE CONVERSIONS

Example 1: Base Conversions

Convert **(105)₁₀** to Binary, Octal, Hex:

- **Binary:** Divide by 2 repeatedly $\Rightarrow$ **(1101001)₂**
- **Octal:** Group binary in 3s **(001 101 001)** $\Rightarrow$ **(151)₈**
- **Hex:** Group binary in 4s **(0110 1001)** $\Rightarrow$ **(69)₁₆**

Example 2: BCD & Excess-3

Convert **(48)₁₀** to BCD and Excess-3:

- **BCD:** Split digits into 4 bits:
4 = 0100, 8 = 1000
 $\Rightarrow$ BCD = **(0100 1000)_{BCD}**
- **Excess-3:** Add 3 to each digit ($4+3=7, 8+3=11$)
 $\Rightarrow$ Excess-3 = **(0111 1011)_{EX-3}**

Example 3: Gray Code

Convert Binary **(101101)₂** to Gray Code:

- $G_5 = B_5 = 1$
- $G_4 = 1 \oplus 0 = 1, G_3 = 0 \oplus 1 = 1$
- $G_2 = 1 \oplus 1 = 0, G_1 = 1 \oplus 0 = 1$
- $G_0 = 0 \oplus 1 = 1$
- $\Rightarrow$ Gray Code = **(111011)_{Gray}**

1.7

ALPHANUMERIC CODES: ASCII & EBCDIC

Alphanumeric codes process both numeric digits and non-numeric alphabetic letters, punctuation marks, and control commands required for computing and data transmission.

Property	ASCII (American Standard Code for Info. Interchange)	EBCDIC (Extended Binary Coded Decimal Interchange)
Bit Length	7-bit standard code (\$2^7 = 128\$ characters); 8-bit extended version (\$2^8 = 256\$).	8-bit fixed code (\$2^8 = 256\$ character representations).
Primary Domain	Universal ANSI standard used across microcomputers, UNIX, PCs, Web.	Proprietary standard developed by IBM for mainframes and AS/400 systems.
Structure	Contiguous alphabetic encoding: 'A'-'Z' (65–90), 'a'-'z' (97–122).	Non-contiguous alphabet blocks (gaps exist between 'I'-'J' and 'R'-'S').
Zone Bits	High 3 bits specify character classification (Control, Symbols, Letters).	First 4 bits specify category class (Digits, Uppercase, Lowercase).

1.8

1'S COMPLEMENT AND 2'S COMPLEMENT ARITHMETIC

Complements simplify binary subtraction inside hardware arithmetic units by transforming subtraction into addition operations.

1's Complement Obtained by replacing every '0' with '1' and every '1' with '0' (bitwise NOT operation). 1's Comp of $(101100)_2 = (010011)_2$ Subtraction Rule: Add 1's complement of subtrahend to minuend. If an end-around carry occurs, add 1 to the LSB of the result.	2's Complement Obtained by adding 1 to the 1's complement of the binary number: $2's\ Comp = 1's\ Comp + 1$. 2's Comp of $(101100)_2 = 010011 + 1 = (010010)_2$ Subtraction Rule: Add 2's complement of subtrahend to minuend. Discard any final end-around carry bit.
---	---

1.9

SIGNED NUMBER REPRESENTATION

For an n -bit binary word, the Most Significant Bit (MSB) acts as the **sign bit** (0 = Positive (+), 1 = Negative (-)). Three major signed systems exist:

Representation System	Positive Range (n -bit)	Negative Range (n -bit)	Zero Representation	Key Characteristics
Sign-Magnitude	0 to $+(2^{n-1} - 1)$	$-(2^{n-1} - 1)$ to -0	Dual Zero (+0 and -0)	MSB is sign; remaining $n-1$ bits represent absolute value magnitude.
Signed 1's Complement	0 to $+(2^{n-1} - 1)$	$-(2^{n-1} - 1)$ to -0	Dual Zero (0000 & 1111)	Negative numbers formed by inverting all bits of positive counterpart.
Signed 2's Complement	0 to $+(2^{n-1} - 1)$	-2^{n-1} to -1	Unique Zero (0000...)	Universally used in hardware CPUs; eliminates dual zero and simplifies ALUs.

Comparative Example: 4-Bit Signed System Range & Values			
Decimal	Sign-Magnitude	Signed 1's Complement	Signed 2's Complement
+7	0111	0111	0111
+1	0001	0001	0001
+0	0000	0000	0000
-0	1000	1111	N/A (Represents 0000)
-1	1001	1110	1111
-7	1111	1000	1001
-8	N/A (Out of range)	N/A (Out of range)	1000